```
SHORTNER-GO PROJECT — ALL SOURCE CODE IN ONE FILE
==================================================
Files are grouped by project layer. Original code is preserved.
Git metadata and go.sum are excluded; go.mod is included for dependencies.


################################################################################
# DB CONNECTION — database/sql.go
################################################################################

package database

import (
      "database/sql"
      "fmt"
      "log"
      "time"

      _ "github.com/jackc/pgx/v5/stdlib"
)

func ConnectDB()*sql.DB{
      dsn:="postgres://postgres:aditya@localhost:5432/goapp"
      db,err:=sql.Open("pgx",dsn)
      if err!=nil{
            log.Fatal(err)
      }

      db.SetMaxOpenConns(25)
    db.SetMaxIdleConns(10)
    db.SetConnMaxLifetime(30 * time.Minute)
    db.SetConnMaxIdleTime(5 * time.Minute)

      err=db.Ping()
      if err!=nil{
            log.Fatal(err)
      }

      fmt.Println("Database Connected")
      return db

}


################################################################################
# REDIS CONNECTION — database/redis.go
################################################################################

package database

import (
      "context"
      "fmt"

      "github.com/redis/go-redis/v9"
)

func ConnectRedis()*redis.Client{
      rdb:=redis.NewClient(&redis.Options{
            Addr: "127.0.0.1:6379",
            PoolSize:     10,
        MinIdleConns: 2,
      })
      _,err:=rdb.Ping(context.Background()).Result()
```

```
      if err!=nil{
            panic(fmt.Sprintf("Failed to connect to Redis: %v", err))
      }
      fmt.Println("Redis connected Sucess")
 return rdb
}


################################################################################
# MODELS — models/url_models.go
################################################################################

package models

import "time"

type URL struct {
      LongUrl  string `json:"longUrl"`
      ShortUrl string `json:"shortUrl"`
      ExpireAt time.Time `json:"expireAt"`
}


################################################################################
# REPOSITORY — repository/shortner_repo.go
################################################################################

package repository

import (
      "crypto/rand"
      "database/sql"

      "math/big"
      "shortner-go/models"
      "time"
)

func generateShortURL() string {
      const chars =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"

      b := make([]byte, 6)

      for i := range b {
            n, err := rand.Int(rand.Reader, big.NewInt(int64(len(chars))))
            if err != nil {
                  panic(err)
            }
            b[i] = chars[n.Int64()]
      }

      return string(b)
}

func Get_short_url(db *sql.DB, shortUrl string) (models.URL, error) {
    var u models.URL

    query := `
        SELECT "shortUrl", "longUrl", "expireAt"
        FROM shortner
        WHERE "shortUrl" = $1
          AND "expireAt" > NOW()
    `
```

```
    err := db.QueryRow(query, shortUrl).Scan(&u.ShortUrl,
        &u.LongUrl,
        &u.ExpireAt,)

    if err != nil {

        return models.URL{}, err

    }

    return u, nil
}

func Create_short(db *sql.DB,longUrl string)(models.URL,error){
      var u models.URL
      u.LongUrl= longUrl
      u.ExpireAt = time.Now().Add(90 * 24 * time.Hour)
      u.ShortUrl = generateShortURL()


      query := `
    INSERT INTO shortner ("shortUrl", "longUrl", "expireAt")
    VALUES ($1, $2, $3)
    RETURNING "shortUrl", "longUrl", "expireAt"
`
      err:=db.QueryRow(
            query,
            u.ShortUrl,
            u.LongUrl,
            u.ExpireAt,
      ).Scan(&u.ShortUrl, &u.LongUrl, &u.ExpireAt)

      if err!=nil{
            return models.URL{},err
      }
      return u,nil
}




################################################################################
# HANDLERS — handlers/shortner_handler.go
################################################################################

package handlers

import (
      "database/sql"
      "encoding/json"
      "fmt"
      "log"
      "net/http"
      "shortner-go/models"
      "shortner-go/repository"
      "time"

      "github.com/redis/go-redis/v9"
)
type Shortner struct {
      DB *sql.DB
      R  *redis.Client
```

```
}
//short url--->it will give u longurl+redirect
func (h Shortner)GetUrl(w http.ResponseWriter,r *http.Request){
  shortUrl:=r.PathValue("shortUrl")

  cached,err:=h.R.Get(r.Context(),shortUrl).Result()

  if err==nil{
       w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(200)
    json.NewEncoder(w).Encode(cached)
       return
  }

//  result,err1:=repository.Get_short_url(h.DB,shortUrl)

//  if err1!=nil{
//    http.Error(w,"No Result found",http.StatusNotFound)
//    return
//  }

//  w.Header().Set("Content-Type", "application/json")
//     w.WriteHeader(200)
//     json.NewEncoder(w).Encode(result.LongUrl)

}

func (h Shortner)CreateUrl(w http.ResponseWriter,r *http.Request){
var u models.URL
json.NewDecoder(r.Body).Decode(&u)

result,err:=repository.Create_short(h.DB,u.LongUrl)
if err!=nil{
      fmt.Println(err)
      http.Error(w,"Error ",http.StatusInternalServerError)
      return
}


err=h.R.Set(r.Context(),result.ShortUrl,result.LongUrl,10*time.Minute).Err()
if err!=nil{
      log.Printf("Redis cache error: %v", err)
      return
}
w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(result)
}


################################################################################
# MAIN / ROUTES — main/main.go
################################################################################

package main

import (
      "fmt"
      "net/http"
      "shortner-go/database"
      "shortner-go/handlers"

)
```

```
/*
var redis1 redis.Client

func hello(w http.ResponseWriter,r *http.Request){
      key:=r.PathValue("key")
      value:=r.PathValue("value")
      redis1.Set(r.Context(),key,value,5*time.Minute)
      fmt.Fprintln(w,"Set sucess")

}

func hello2(w http.ResponseWriter,r *http.Request){
      key:=r.PathValue("key")
      val,err:=redis1.Get(r.Context(),key).Result()
      if err!=nil{
            log.Println(err)
            return
      }
      fmt.Fprintln(w,val)

}*/

func main(){
 db:=database.ConnectDB()
 defer db.Close()

 redis:=database.ConnectRedis()
 defer redis.Close()

 mux:=http.NewServeMux()

handler:=&handlers.Shortner{
      DB: db,
      R: redis,
}

  mux.HandleFunc("GET /v1/url/{shortUrl}",handler.GetUrl)
 mux.HandleFunc("POST /v1/url/",handler.CreateUrl)



 server:=&http.Server{
      Addr: ":8000",
      Handler: mux,
 }

 fmt.Println("Server is live at port 8000")
 server.ListenAndServe()


}


################################################################################
# GO MODULE / DEPENDENCIES — go.mod
################################################################################

module shortner-go

go 1.27.0

require (
      github.com/cespare/xxhash/v2 v2.3.0 // indirect
      github.com/jackc/pgpassfile v1.0.0 // indirect
```

```
	github.com/jackc/pgservicefile v0.0.0-20240606120523-5a60cdf6a761 //
indirect
	github.com/jackc/pgx/v5 v5.11.0 // indirect
	github.com/jackc/puddle/v2 v2.2.2 // indirect
	github.com/redis/go-redis/v9 v9.22.0 // indirect
	go.uber.org/atomic v1.11.0 // indirect
	golang.org/x/sync v0.17.0 // indirect
	golang.org/x/sys v0.30.0 // indirect
	golang.org/x/text v0.29.0 // indirect
)
```